



PHASE 6

Data Model and Security

Schema, authorization, security controls and secret handling

SMARTPOCKET AI • SMART BUDGET AND RECOMMENDATION ASSISTANT

1. Data Model and Security

1.1 Data Model

Entity	Key fields	Purpose
User	id, name, username, email, password_hash, created_at	Represents an authenticated account.
HistoryEntry	id, user_id, type, title, input, result, created_at	Stores a planner request and its normalized response.

The relationship is one user to many history entries. History access is always filtered by the authenticated user identifier. JSON fields preserve the original form and the complete normalized result so a plan can be reviewed or reused.

1.2 Security Controls

- Passwords are stored as bcrypt hashes rather than plaintext.
- JWT signatures protect token integrity and include an expiration time.
- Planner and history routes require a valid authenticated user.
- Object-level checks prevent access to another user's history entries.
- Pydantic rejects invalid types, lengths, and ranges before business logic runs.
- CORS is restricted to configured frontend origins.
- Gemini keys and JWT secrets remain outside source files in environment variables.

Secret handling

Never place API keys, JWT secrets, or real credentials in screenshots, source examples, repositories, or documentation. Rotate any key that has been exposed.

History ownership query

```
entry = (  
    db.query(HistoryEntry)  
    .filter(  
        HistoryEntry.id == entry_id,  
        HistoryEntry.user_id == current_user.id,  
    )  
    .first()  
)  
if entry is None:  
    raise HTTPException(status_code=404, detail="History entry not found")
```